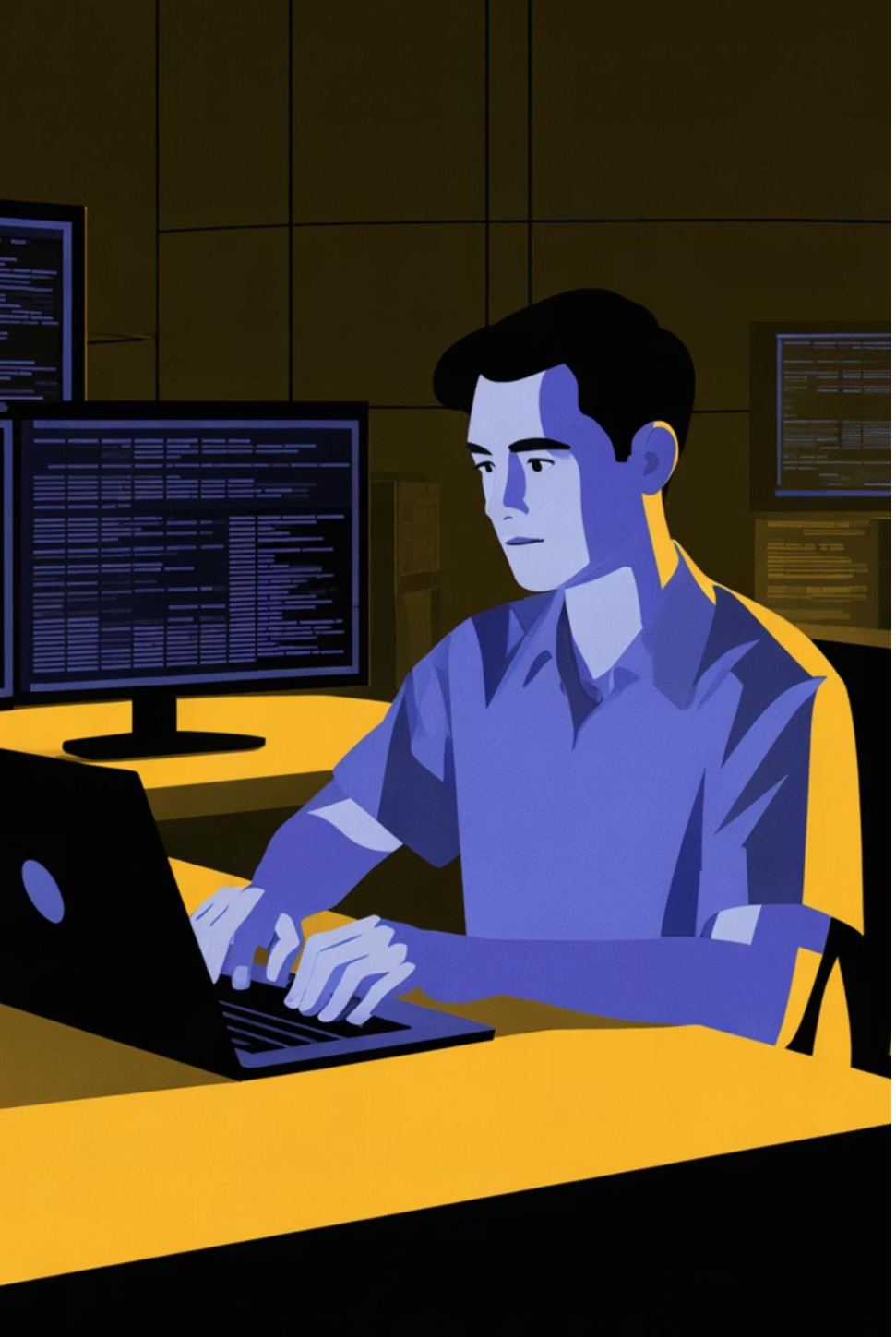
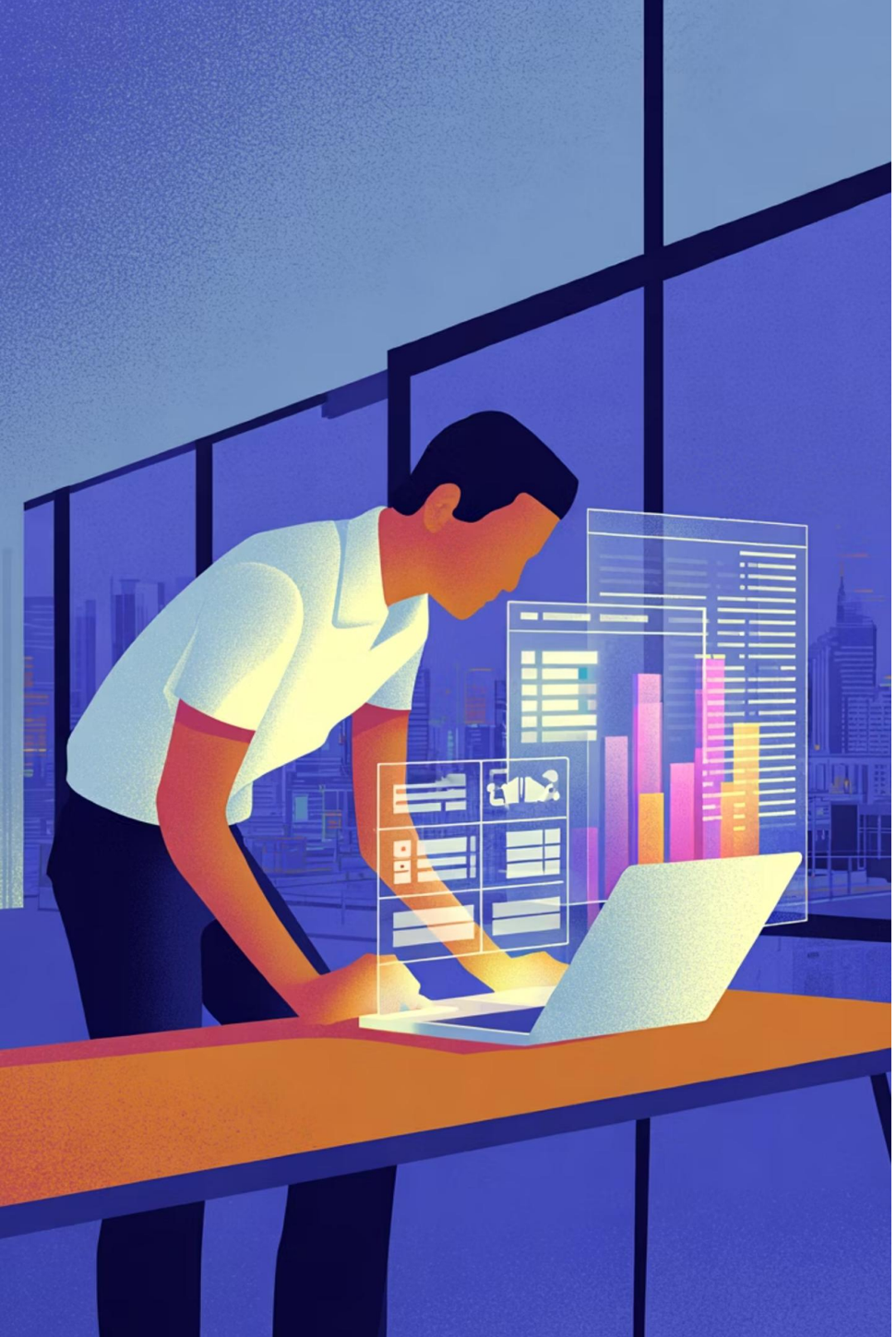


SQL Intermediate Query Practice

Real Interview Questions with Solutions

MAYANK BACHCHANI



Why SQL Practice Matters

SQL in the Real World

SQL remains one of the most in-demand skills across data analysis, backend development, and business intelligence roles. Nearly every technical interview for a data-related position includes SQL challenges.

Practice with Purpose

Solving real interview-style queries builds pattern recognition, sharpens syntax recall, and boosts confidence under pressure. This deck covers 10 targeted questions to prepare you for what interviewers actually ask.

Dataset Overview

All queries in this presentation use a single table: **employees**. Familiarize yourself with its structure before diving in.

Column	Type	Description
emp_id	INT	Unique employee identifier
emp_name	VARCHAR	Full name of the employee
department	VARCHAR	Department the employee belongs to
salary	DECIMAL	Monthly salary amount
city	VARCHAR	City of work location
join_date	DATE	Date the employee joined

All 10 practice queries operate on this **employees** table. Keep this schema handy as your reference.

CORRELATED SUBQUERY

Employees Above Department Average Salary

What It Tests

A **correlated subquery** runs once per row in the outer query. Here, for each employee, it dynamically calculates the average salary of *their own department* and compares it to that individual's salary.

Key insight: `e1.department = e2.department` is the correlation — it links the outer and inner queries row by row.

Query

```
SELECT *
FROM employees e1
WHERE salary > (
 SELECT AVG(salary)
 FROM employees e2
 WHERE e1.department = e2.department
)
ORDER BY department;
```

NESTED SUBQUERY

Second Highest Salary in the Company

Query

```
SELECT emp_name, salary
FROM employees
WHERE salary = (
  SELECT MAX(salary)
  FROM employees
  WHERE salary < (
    SELECT MAX(salary)
    FROM employees
  )
);
```

How It Works

This uses **two nested subqueries**. The innermost query finds the highest salary. The middle query finds the highest salary *below* that value — which is the second highest. The outer query returns all employees earning that amount.

⚠ Watch out: If multiple employees share the second highest salary, all of them are returned.

🏆 WINDOW FUNCTION + CTE

Top 2 Highest-Paid Employees per Department

Query

```
WITH mycte AS (
  SELECT emp_name, department, salary,
    DENSE_RANK() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS rankk
  FROM employees
)
SELECT *
FROM mycte
WHERE rankk <= 2;
```

Key Concepts

CTE (WITH)

Creates a reusable named result set for cleaner logic.

DENSE_RANK()

Ranks without gaps — ties both get rank 1, next gets rank 2.

PARTITION BY

Resets the ranking for each department independently.

AGGREGATION

Department-Wise Total & Average Salary

Query

```
SELECT department,
  SUM(salary) AS total,
  ROUND(AVG(salary), 2) AS
avg_
FROM employees
GROUP BY department;
```

What to Know

GROUP BY collapses all rows with the same department into one. **SUM** adds all salaries, while **ROUND(AVG(), 2)** gives a clean two-decimal average. A classic aggregation pattern you'll see in nearly every SQL interview.

FILTERING

Employees Hired After 2021 with Salary > 50,000

Query

```
SELECT *
FROM employees
WHERE join_date > '2020-12-31'
  AND salary > 50000;
```

Filter Logic

Two **WHERE** conditions combined with **AND** — both must be true for a row to be included. Note that `'2020-12-31'` is used instead of `'2021-01-01'` to ensure all 2021 and later dates are captured regardless of the data format.

> Always verify date formats match your database's expected type — `DATE`, `DATETIME`, or `VARCHAR`.

HAVING CLAUSE

Cities with More Than One Employee

Query

```
SELECT city, COUNT(*)
FROM employees
GROUP BY city
HAVING COUNT(*) > 1;
```

WHERE vs HAVING

This is a common interview trap. **WHERE** filters individual rows *before* grouping. **HAVING** filters groups *after* aggregation. Since we're filtering on a `COUNT()`, we must use **HAVING** — not WHERE.

Common mistake: Using WHERE with aggregate functions causes a syntax error. Always use HAVING post-GROUP BY.

WINDOW FUNCTIONS

Advanced Window Function Queries

Highest Salary per Department

```
SELECT department,
  MAX(salary) AS maximum
FROM employees
GROUP BY department;
```

Simple aggregation using **GROUP BY + MAX()** to find the top earner per department.

Salary Difference from Max

```
SELECT emp_name, salary,
  MAX(salary) OVER() AS
maximum,
  (MAX(salary) OVER() -
salary) AS diff
FROM employees;
```

MAX() OVER() without PARTITION BY applies globally — showing each employee's gap from the company's top salary.

Rank All Employees by Salary

```
SELECT emp_name,
department, salary,
  DENSE_RANK() OVER(
    ORDER BY salary DESC
  ) AS rankk
FROM employees;
```

Company-wide ranking using **DENSE_RANK()** — no gaps when salaries tie.